SIGH — Propuesta del proyecto

Sistema Integrado de Gestión de Horarios (SIGH). Proyecto final de Bases de Datos 1, Ingeniería de Sistemas, Universidad El Bosque.

Equipo: Julián Felipe López Narváez, David Alejandro Ordaz Luque y Ángel Daniel Aya Páez.

El SIGH es una base de datos relacional que guarda la programación semanal de horarios de clases de un programa de pregrado y hace cumplir sus reglas en el propio motor: la base de datos impide guardar un horario con cruces, sobrecupos o docentes por encima de su contrato. Es un sistema genérico; no depende de personas, cargos ni normas de una institución concreta.

1. Problemática

Cada semestre, un programa de pregrado debe programar los horarios semanales de sus cursos: decidir qué docente dicta cada sección, en qué espacio y en qué franja. Cuando esa programación se hace en hojas de cálculo desconectadas de los demás sistemas institucionales, aparecen estos problemas:

- Cruces: un docente, un espacio o una sección con dos clases a la misma hora.
- Secciones asignadas a espacios con menos capacidad que su matrícula o sin los recursos que el curso exige.
- Docentes con más horas de clase de las que permite su tipo de contrato.
- Clases programadas en horas no permitidas, como la franja de 12:00 a 13:00.
- Secciones con muy pocos estudiantes que nadie detecta a tiempo para cerrarlas.
- Ningún registro de quién cambió el horario ni de qué versión fue la aprobada.

El problema es una instancia del *timetable problem*, un problema de planificación sin solución algorítmica definitiva. Este proyecto no busca generar el horario óptimo: busca que los datos del horario sean consistentes, es decir, que la propia base de datos impida guardar un horario que viole las reglas del dominio.

2. Objetivos

Objetivo general. Diseñar e implementar una base de datos relacional en PostgreSQL que almacene la programación semanal de horarios de clases de un programa de pregrado y haga cumplir las reglas del dominio en el propio motor, accesible desde una aplicación web.

Objetivos específicos.

1. Modelar el dominio con un MER en notación de Chen, su versión extendida y su representación en Crow's Foot.
2. Transformar el MER en un modelo relacional normalizado al menos hasta la Tercera Forma Normal.
3. Implementar la base de datos en PostgreSQL sobre una máquina virtual Linux, con restricciones, disparadores, vistas, roles y datos de prueba.

4. Formular al menos 50 consultas en álgebra relacional sobre el esquema y traducirlas a SQL.
5. Conectar una aplicación web (backend y frontend) que lea y escriba en la base de datos.
6. Documentar el proceso y gestionar el código y la documentación en un repositorio de GitHub con aportes de los tres integrantes.

3. Alcance y exclusiones

Dentro del alcance:

- Catálogos de períodos académicos, tipos de contrato, docentes, cursos, espacios (aulas, laboratorios de cómputo y laboratorios especializados), recursos, franjas horarias y parámetros del sistema.
- Apertura y cierre de secciones de cada curso por período, con cupos y matrícula.
- Disponibilidad horaria de cada docente por período.
- Versiones del horario y asignación de sesiones (sección, docente, espacio y franja), con validación de reglas en la base de datos.
- Flujo de aprobación de una versión: En edición, Pendiente, Avalado y Sincronizado.
- Consultas y reportes de horario por docente, espacio y sección; carga de los docentes; secciones en riesgo de cierre.
- Usuarios con roles, permisos por rol y bitácora de auditoría de solo inserción.
- Aplicación web que lee y escribe en la base.

Fuera del alcance:

- Algoritmo de generación u optimización automática del horario.
- Integración automática con sistemas externos: el Administrador registra docentes y espacios por script o por una API de ingesta, la matrícula se carga con scripts y la sincronización documental solo registra un identificador.
- Autenticación real (contraseñas de prueba, sin proveedor de identidad).
- Gestión individual de estudiantes e inscripción de materias: la matrícula llega agregada por sección.
- Notificaciones, despliegue en producción y nómina docente.

4. Actores y solución propuesta

Actor	Tipo	Responsabilidad
Administrador	Rol del sistema	Mantiene catálogos, parámetros, usuarios y roles; carga datos externos; registra la sincronización documental
Coordinador académico	Rol del sistema	Gestiona cursos, secciones y versiones; asigna sesiones; autoriza sobrecupos; envía versiones a aprobación
Aprobador	Rol del sistema	Avala o devuelve las versiones del horario
Docente	Rol del sistema	Registra su propia disponibilidad y consulta su

Actor	Tipo	Responsabilidad
		horario
Sistemas fuente externos	Sistema externo	Proveen los datos de docentes, espacios y matrícula; el Administrador los registra en el sistema
Repositorio documental externo	Sistema externo	Archiva el horario avalado y devuelve un identificador
Estudiantes	Interesado indirecto	Consumen el horario publicado; no usan el sistema

Solución propuesta. La base de datos corre en PostgreSQL sobre una máquina virtual Linux en la nube. Las reglas del dominio se implementan en el motor con llaves, restricciones UNIQUE y CHECK, disparadores y permisos por rol; la aplicación web (backend y frontend) solo muestra los rechazos que la base devuelve. El diseño sigue el camino MER → MER extendido → modelo relacional → normalización → implementación física, y cada paso tiene su propio documento en el repositorio.